AiNspire

TM AI PERSONA ASSESSMENT PLATFORM

DOCUMENT 1 OF 2 · TECHNICAL BASELINE

Full System Functionality Specification

A complete, screen-by-screen and endpoint-by-endpoint account of what the AiNspire v1 platform does, how each function works internally, and an explicit separation of which capabilities are backed by live logic versus demonstration data.

Codebase
ainspire-dev / v1

Repository
shahrilnizam-25/ainspire-dev-v1

Commit at review
27ebc07 (main)

Prepared for
Shahril Nizam · Tien · Ain

Org context
Group IT & Digital (GITD)

Date
1 October 2026

— Contents

—	**How to Read This Document**	Start here · glossary of every term
1	**System Overview**	What the product is and who uses it
2	**Architecture & Deployment Topology**	Runtime, hosting, data flow
3	**Repository & Module Map**	Every package and its role
4	**Domain Model**	Personas, questions, languages
5	**Backend Functional Specification**	All three API endpoints in detail
6	**Frontend Functional Specification**	All nine screens, state and logic
7	**Cross-Cutting Mechanisms**	i18n, caching, fallback, export
8	**Live vs Simulated — Reality Register**	Critical demo-integrity reference
9	**Defect & Gap Register**	Confirmed issues found in review
10	**Prototype Backlog (mockup-sandbox)**	Designed but unbuilt features

— How to Read This Document

THE WHOLE DOCUMENT IN SIX SENTENCES

AiNspire asks an employee six questions, sends the answers to TM's own AI, and the AI decides which of four "AI personas" that person is. The employee gets a written explanation, three learning suggestions, and a downloadable PDF report. HR gets a dashboard showing the whole team's personas, plus an AI-written 90-day upskilling plan. **The AI part is genuinely real. Almost everything else on the HR and statistics screens is demonstration data that was typed into the code by hand.** Nothing the user does is ever saved — close the browser and it is gone.

Who should read what

If you are…	Read these sections	Why
Non-technical (business, pitching, selling)	1, 8, 9, 10	Section 1 explains what it does. **Section 8 is the one to memorise** — it tells you exactly what is real and what is fake, so you never over-claim in front of a client. Section 9 is the to-do list.
Design / front end	1, 6, 8, 10	Section 6 walks through all nine screens. Section 10 shows the six designs that were drawn but never built.
Engineering	All of it	Sections 2–7 are the internals.

What the coloured labels mean

Label	Meaning
LIVE	Really works. A real AI call, a real file download, real calculations.
DERIVED / STATIC / PARTIAL	Half-real. The number is calculated from something real, but it was not independently measured — or the text was written in advance rather than generated.
SIMULATED / COSMETIC / NONE	Not real. The data was typed into the code, or the button only changes its own label and does nothing else.
CRITICAL HIGH MEDIUM/LOW	How urgent a problem is. Critical means fix it before anyone else sees the system.

Plain-English glossary

Every technical term used in this document, explained once. `Text in this grey style` is the literal name of a file, a setting or a piece of code.

Term	What it actually means
Frontend	Everything you see and click in the browser. The screens.
Backend / API server	A small program running on a server. The browser sends it questions; it answers. The user never sees it.
Endpoint / route	One specific thing the backend can be asked to do — like one phone extension. AiNspire has three.
LLM	Large Language Model — the AI. Ours is TM's own, hosted inside TM, not ChatGPT.
Prompt	The instructions we write and send to the AI along with the employee's answers. Our prompt is where the product's real intelligence lives.
Token	Roughly a word. "Max tokens 1024" means "don't write more than about 750 words back".
Hardcoded	Typed directly into the code by a developer. It never changes and it did not come from a database or a real person.
Persist / persistence	Saving data so it still exists tomorrow. AiNspire has none.
Database / schema	Where saved data would live, and the list of what gets saved. Ours is an empty shell.
State	What the app is currently remembering, in browser memory only. Refreshing the page erases it.
Cache	Keeping an answer we already fetched so we don't have to ask again. We cache one AI result per language.
Fallback	The backup plan when the AI fails — we simply count which persona was picked most often.
Validation	Checking that incoming or outgoing data has the right shape before trusting it.
Authentication / SSO	Logging in and proving who you are. SSO means using your normal TM account. We have neither.
CORS	A browser rule about which websites are allowed to call our backend. Ours currently allows everyone.

Term	What it actually means
Rate limiting	Capping how many times someone can call our backend, so nobody can run up TM's AI bill.
API key	A password that lets our server use TM's AI. Ours is currently visible in public code — see Section 5.2.
Deployment / hosting	Putting the app on the internet. Our screens and our backend live on two different services.
Repository (repo)	The folder of code, stored on GitHub.
Environment variable	A setting supplied from outside the code — used for passwords and addresses so they are not written into the code itself.
Mockup / prototype	A screen that was designed and built to look at, but was never connected to anything.

1 System Overview

AiNspire is an internal workforce AI-readiness platform for Telekom Malaysia. It serves two distinct user groups through a single React single-page application.

1.1 The employee journey

An employee answers five multiple-choice questions and one open-ended question, and states their current role at TM. All answers are posted to a TM-hosted large language model, which reasons across the whole submission and returns a single **AI persona** classification, a confidence score, written reasoning, a personalised narrative, and three tailored learning recommendations. The employee can then view a formatted one-page report and download it as a PDF.

1.2 The HR manager journey

An HR manager opens a dashboard showing team-level persona distribution, individual results, a skills-gap analysis tool, a leadership succession pipeline, and an AI-generated 90-day upskilling action plan. A separate organisation-wide statistics screen presents workforce analytics across ten TM divisions.

1.3 The four AI personas

Persona	Colour	Definition sent to the LLM
The AI Explorer `explorer`	`#00D4FF Cyan`	Curious and experimental — loves discovering AI tools, early adopter mindset, enthusiastic but without deep technical depth.
The AI Builder `builder`	`#4361EE Blue`	Technically hands-on — implements and integrates AI solutions, builds AI-powered systems, high technical depth.
The AI Strategist `strategist`	`#7B2FBE Purple`	Business-aligned — leads AI projects, drives measurable ROI, bridges technical teams and executive stakeholders.
The AI Visionary `visionary`	`#FFB700 Gold`	Transformational — shapes long-term AI direction, inspires others, influences policy and thought leadership at industry level.

DESIGN PRINCIPLE WORTH NOTING

The persona definitions exist in **two places with different wording**: a short behavioural definition in `classify.ts` that is injected into the LLM prompt, and a longer employee-facing description in `personas.ts` used for display. The LLM only ever sees the former. This is deliberate — the model is given tight discriminative criteria, while the user is shown aspirational copy.

2 Architecture & Deployment Topology

2.1 Runtime stack

Layer	Technology	Notes
Frontend	React 19, Vite 7, TypeScript, Tailwind CSS v4	Single-page app; screen routing by local state, not a router
Animation	Framer Motion	Used on every screen transition and element entrance
Charts	Recharts 2.15	Area, Bar, Line, Pie — statistics screen only
Backend	Express 5, Node.js 24, TypeScript	Three routes, no database connection in use
Bundling	esbuild (`build.mjs`) for API; Vite for frontend	API emits a single `dist/index.mjs`
Logging	Pino + pino-http, pino-pretty in dev	Structured JSON logs with secret redaction
LLM	TM-hosted OpenAI-compatible endpoint, model `gpt-oss-20b`	Reached over the TM GPU proxy
Monorepo	pnpm workspaces + catalog protocol	Packages under `artifacts/` , `lib/` , `scripts/`
PDF	html-to-image + jsPDF (client-side)	Chosen over html2canvas to survive modern CSS colours

2.2 Deployment

The system is split across two hosting providers, which is the single most important operational fact about it.

IN PLAIN WORDS

The screens and the AI server live in **two completely different places on the internet**. The screens are on GitHub Pages; the server is on a service called Render.

Think of it as a shopfront on one street and the kitchen on another. The shopfront has to know the kitchen's address. If we build the shopfront without writing that address down, it quietly starts serving from an empty cupboard instead of telling anyone there's a problem.

Component	Host	Mechanism
Frontend	GitHub Pages `shahrilnizam-25.github.io/ainspire-dev-v1/`	GitHub Actions workflow `.github/workflows` triggers on push to `main`. Builds `artifacts/tm-ai-persona` with `BASE_PATH=/ainspire-dev-v1/` and injects `VITE_API_URL` from repository secrets. Adds `.nojekyll` so Jekyll does not strip Vite's underscore-prefixed asset folder.
API server	Render (`render.yaml` , service `ainspire-api`)	Installs pnpm at workspace root, runs `node build.mjs` , starts `dist/index.mjs` on port 10000. `TM_LLM_API_KEY` is marked `sync: false` , meaning it must be set manually in the Render dashboard.

CONSEQUENCE OF THIS SPLIT

Because the frontend is a static site on GitHub Pages and the API lives on a separate origin, every AI call is a **cross-origin request**. The Express app therefore enables `cors()` with no allow-list, and the frontend must be built with a correct `VITE_API_URL`. If that secret is missing or wrong at build time, the deployed site silently falls back to same-origin `/api/classify`, which does not exist on GitHub Pages — the app will then show the rule-based fallback persona instead of an AI result. This is the most likely cause of a failed live demo.

What to actually check before a demo: open the deployed site, complete an assessment, and confirm the results badge says **"AI classified"** in cyan and not the yellow **"rule-based"** badge. Yellow means the AI was never reached.

2.3 End-to-end request flow

```
EMPLOYEE CLASSIFICATION PATH

  Browser (GitHub Pages)
       │  5 MCQ answers + role + free text + lang
       ▼
  POST {VITE_API_URL}/api/classify
       │
       ▼
  Express 5 on Render
       │  pino-http request log → request_started → classify_started
       ├─ validate: answers[] present and non-empty, else 400
       ├─ partition: MCQ items / role item (questionId 6) / open item
       ├─ assemble single-shot prompt (persona defs + role + answers + language rule)
       ▼
  POST {TM_LLM_BASE_URL}/chat/completions  model=gpt-oss-20b  max_tokens=1024
       │
       ▼
  Raw string response
       ├─ strip markdown code fences
       ├─ JSON.parse
       ├─ sanitise: persona in enum? confidence numeric? recommendations array?
       ▼
  200 { persona, confidence, reasoning, narrative, recommendations[] }
       │
       ▼
  React caches result per language → Results screen → Report screen → client-side PDF

On any throw: 500 { error, details } → frontend falls back to plurality-vote persona
```

3 Repository & Module Map

IN PLAIN WORDS

The code is organised into separate folders, each with a job. Two of them are the real product (the screens and the server). The rest are either design experiments or half-finished plumbing that nothing currently uses.

We built the shop and the kitchen, then built a very nice stockroom, a delivery system and a filing cabinet — and never plugged any of them in. The good news is that they are already built.

Path	Status	Purpose and current reality
`artifacts/tm-ai-persona`	ACTIVE	The production React application. Nine screens, trilingual, ~3,900 lines of first-party code plus a full shadcn/ui component library.
`artifacts/api-server`	ACTIVE	Express 5 API. Three routes, one shared LLM helper, one logger. Roughly 260 lines total — deliberately thin.
`artifacts/mockup-sandbox`	DESIGN ONLY	A self-contained Vite app that auto-discovers and previews design prototypes. Contains six mockups representing the intended product roadmap. Not deployed.
`lib/api-spec`	STALE	OpenAPI 3.1 specification plus an Orval codegen config. Documents only `/healthz` and `/classify`; omits `/action-plan` and the `lang` parameter, and still describes the model as "Claude".
`lib/api-client-react`	UNUSED	Orval-generated React Query hooks and a custom fetch wrapper. The application imports none of it — every call is a hand-written `fetch`.
`lib/api-zod`	PARTLY USED	Generated Zod schemas. Only `HealthCheckResponse` is actually consumed, inside the health route. Classification responses are never validated with Zod.
`lib/db`	EMPTY	Drizzle ORM wiring to PostgreSQL exists, but `schema/index.ts` declares no tables — it is still the scaffold's commented template. **The platform stores nothing.**
`scripts`	SCAFFOLD	Contains only a `hello.ts` placeholder and a post-merge hook.
`attached_assets`	—	~68 source images and reference material carried over from the Replit build.

ARCHITECTURAL REALITY CHECK

The repository contains a well-formed contract-first API pipeline — OpenAPI spec → Orval → typed hooks and Zod schemas — that the application **bypasses entirely**. Three of the four `lib/` packages are effectively dead weight at runtime. This matters for Document 2: the scaffolding to do this properly already exists and only needs to be connected, which is a high-value, low-effort improvement.

4 Domain Model

4.1 The assessment instrument

Six questions are asked in total. Questions 1–5 are multiple choice with four options each; every option is pre-mapped to exactly one persona. Question 6 is free text, preceded by a mandatory role field.

Q	Type	Subject	Mapping
1	MCQ	How you currently engage with AI tools in daily work	Option A → explorer Option B → builder Option C → strategist Option D → visionary
2	MCQ	Your AI knowledge level	
3	MCQ	Your natural role with AI when the team faces a challenge	
4	MCQ	What motivates you most about AI at TM	
5	MCQ	How you typically learn about AI	
6a	Short text	Your current role at Telekom Malaysia — **required**	Context only
6b	Long text	What AI means for your role and where you want to take your AI journey — **minimum 20 characters**	Weighted heavily by the LLM

4.2 Languages

Three languages are supported end to end: **EN** (English), **BM** (Bahasa Melayu) and **CN** (Simplified Chinese). The i18n module is a single 1,310-line TypeScript file holding three parallel dictionaries, plus fully translated copies of all six questions and every answer option. There is no translation library and no runtime loading — everything is compiled into the bundle.

Language affects three things: the UI chrome, the questions presented, and — critically — the LLM output itself. When the selected language is not English, a hard instruction is appended to the prompt requiring every generated string to be written in that language with no English mixed in.

5 Backend Functional Specification

> **IN PLAIN WORDS — WHAT THIS WHOLE SECTION COVERS**
>
> The server can only do three things. It can say "I'm alive", it can classify an employee, and it can write a 90-day plan for HR. That is the entire backend — about 260 lines of code. Everything else in the product happens inside the browser.

5.1 Application assembly — `app.ts`

The Express instance is built in a fixed middleware order:

1. **pino-http** with custom serialisers. The request serialiser deliberately strips the query string (`req.url.split("?")[0]`) so query parameters never reach the logs; the response serialiser emits only the status code.
2. **cors()** with default options — currently **all origins allowed**.
3. **express.json()** and **express.urlencoded({ extended: true })**.
4. A custom timing middleware that logs `request_started` (with method, path, origin and user-agent) and, on response `finish`, logs `request_finished` with the status code and elapsed milliseconds.
5. All routes mounted under the `/api` prefix.

The logger (`lib/logger.ts`) redacts `authorization` and `cookie` request headers and any `set-cookie` response header, and switches to pretty-printed colourised output whenever `NODE_ENV` is not `production`.

The entrypoint (`index.ts`) refuses to start without a valid numeric `PORT`, throwing on a missing or non-positive value rather than silently defaulting.

5.2 The LLM helper — `lib/llm.ts`

> **IN PLAIN WORDS**
>
> This is the one small piece of code — about 26 lines — that actually talks to TM's AI. Every AI feature in the entire product goes through it. It has no time limit and no retry, so if TM's AI is slow, the whole request simply waits. And it contains the security problem described below.

A single exported function, `chatComplete(prompt, maxTokens = 1024)`, is the only path to the model in the entire system. It performs one `POST` to `{BASE_URL}/chat/completions` with a single user-role message, throws an `Error` containing the HTTP status and body text if the response is not OK, and otherwise returns `choices[0].message.content`, falling back to the literal string `"{}"` if that path is absent.

Three environment variables control it, each with a code default: `TM_LLM_BASE_URL`, `TM_LLM_API_KEY` and `TM_LLM_MODEL`.

> **CRITICAL SECURITY FINDING**
>
> The default value for `TM_LLM_API_KEY` is a **real, hardcoded TM API key committed into the source file** and pushed to a public GitHub repository. It is present in the current `main` branch and in the full commit history. Anyone who finds the repository can call the TM GPU proxy with TM's credential.
>
> There are also no timeout, no retry, and no abort controls on this fetch. A hung TM proxy will hold the Express request open indefinitely. The frontend has a 45-second client-side abort, but only on the language-switch path — not on the initial classification.

5.3 `GET /api/healthz`

Returns `{ "status": "ok" }`, parsed through the generated Zod `HealthCheckResponse` schema before being sent. This is the only place in the codebase where generated validation is actually exercised.

5.4 `POST /api/classify` — persona classification

> **IN PLAIN WORDS**
>
> This is the heart of the product. It takes the six answers, wraps them in a carefully written instruction to the AI, sends it off, and tidies up whatever comes back so the screens never break.
>
> *Imagine handing an experienced assessor a candidate's form along with a one-page briefing on what the four personas mean and what to look out for. The briefing is the prompt — and writing a good briefing is the actual skill here.*

Request

`{ answers: AnswerItem[], lang?: "EN" | "BM" | "CN" }`, where each `AnswerItem` carries `questionId`, `questionText`, and then either MCQ fields (`selectedOption`, `selectedText`, `personaId`) or a `freeText` field.

Processing steps

1. **Log** `classify_started` with the answer count and language.
2. **Validate.** If `answers` is missing, not an array, or empty, return `400 { error: "answers array is required" }`. Note this is the *only* validation — individual item shapes are never checked.
3. **Partition the answers** by structure rather than by index:
   - MCQ items = those with a defined `selectedOption`.
   - The role item = the item with `freeText` **and** `questionId === 6`.
   - The open-ended item = the item with `freeText` and any other `questionId`.
4. **Build the prompt.** A single-shot prompt is assembled containing: an expert-role framing ("AI talent classifier for Telekom Malaysia's workforce development programme"), the conditional language directive, the four persona definitions, the employee's stated role, every MCQ rendered as question text plus selected option text plus its mapped persona, the open-ended response, six explicit reasoning instructions, and a literal JSON output template.
5. **Call the model** via `chatComplete(prompt, 1024)`.
6. **Clean the response.** Strip ```` ```json ```` and ```` ``` ```` fences with a global regex, then trim.
7. **Parse and sanitise.** `JSON.parse`, then three defensive corrections: an out-of-enum `persona` is forced to `"explorer"`; a non-numeric `confidence` becomes `0.7`; a non-array `recommendations` becomes `[]`.
8. **Log** `classify_success` with the chosen persona and confidence, and return the object.

The six reasoning instructions given to the model

These are the heart of the product's differentiation and are worth stating verbatim in substance: analyse role, MCQs and free text *together* as a whole picture; factor in the employee's role as context for likely AI exposure; **pay special attention to the open-ended response because it reveals true orientation more than MCQ choices**; expect and resolve mixed signals with professional judgement; assign the single best-fitting persona rather than the plurality vote; and cite actual phrases from the answers in the reasoning.

> **WHY THIS IS THE PRODUCT'S REAL DIFFERENTIATOR**
>
> A naive quiz would simply count which persona was selected most often. This system explicitly instructs the model to **override the plurality vote** when the free-text response contradicts it. That is the difference between a personality quiz and an assessment — and it is the one thing in the demo that is unambiguously real.
>
> **Say it like this in a pitch:** "A quiz counts your answers. AiNspire reads them. If you tick the beginner boxes but then describe building an automation that saved your team six hours a week, it will classify you as a Builder and tell you exactly which sentence changed its mind."

Error behaviour

Any throw — network failure, non-OK LLM status, or malformed JSON — is caught, logged as `classify_failed`, and returned as `500 { error: "AI classification failed", details: String(err) }`. The `details` field stringifies the raw error, which on an LLM HTTP failure includes the upstream response body.

5.5 `POST /api/action-plan` — HR 90-day upskilling plan

Request

`{ distribution, teamSize, divisionName, dominantPersona, skillsGap[] }` where `distribution` maps persona to a percentage and each `skillsGap` entry holds `{ persona, current, target, gap }`.

Processing

The gap list and distribution are flattened into readable bullet summaries and embedded in a prompt that frames the model as an "AI Workforce Development Strategist for Telekom Malaysia". The model is instructed to produce exactly three phases (days 1–30, 31–60, 61–90), each with a title, a one-sentence objective, three to four *concrete* actions, and a measurable success metric — and to tailor those actions to the specific gaps, with a worked example given in the prompt ("if Explorer % is above target, recommend structured implementation programmes"). `max_tokens` is raised to **2048** for this call.

Response shape

`{ planTitle, executiveSummary, phases[{ phase, title, days, objective, actions[{ action, owner, effort, personas[] }], successMetric }], keyRisks[], expectedOutcome }`

> **VALIDATION ASYMMETRY**
>
> Unlike `/classify`, this route performs **no input validation at all**. It immediately calls `skillsGap.map(...)`, so a request without a `skillsGap` array throws a `TypeError` before any logging, which is then surfaced to the client as a generic 500. It also performs no output sanitisation — whatever JSON the model returns is passed straight through to the React renderer.

6 Frontend Functional Specification

IN PLAIN WORDS — WHAT THIS WHOLE SECTION COVERS

Nine screens. Five of them are the employee's journey (welcome → questions → open question → AI thinking → result → report). Two are for HR (dashboard, statistics). Two are supporting (contact, and the printable report). This section explains each one and, where relevant, says plainly whether what you see is real.

6.1 Shell and navigation — `App.tsx`

IN PLAIN WORDS

The app is a single page that swaps one screen for another, rather than a website with real pages and web addresses. That means **there is no URL you can share for a specific result**, and pressing the browser's Back button does not go back a screen — it leaves the app. Worth knowing before a demo.

There is no router. A single `screen` state variable of a nine-value union (`landing | assessment | open-question | ai-loading | results | hr-view | statistics | contact | report`) drives a Framer Motion `AnimatePresence` block in `mode="wait"`, so each screen fully exits before the next enters. A fixed language switcher sits at the top right on every screen, with an animated pill that slides between EN / BM / CN using a shared `layoutId`.

State held at the root

State	Role
`screen`	Which of the nine screens is mounted
`lang`	Active language; drives both UI copy and LLM output language
`currentQuestionIdx`	Position within the five MCQs
`mcqAnswers`	Accumulated MCQ answers with their persona mappings
`aiResultCache`	**Per-language cache** of classification results — `Partial<Record<Lang, AIResult>>`
`classifiedPersonaId`	The canonical persona, set once from the first successful classification
`lastAnswers`	The exact payload sent, retained so a language switch can re-request
`userRole`	Free-text role, used in the report header and PDF filename
`isReClassifying`	Drives the "Updating content…" badge and dimming overlay
`fetchingForLang`	A `useRef` guard preventing duplicate in-flight requests for the same language

The language-switch re-classification mechanism

> **IN PLAIN WORDS**
>
> If you switch from English to Bahasa Melayu after seeing your result, the app asks the AI to write everything again in Malay. But it deliberately **keeps your original persona** — only the wording changes, never the verdict. It also remembers each language so switching back and forth is instant, and it gives up after 45 seconds rather than spinning forever.

This is the most intricate logic in the frontend and is worth understanding precisely. When the user changes language while on the Results or Report screen, an effect fires that: returns immediately on a cache hit; aborts if a request for that same language is already in flight; otherwise starts a fresh classification with the stored answers, guarded by an `AbortController` and a **45-second hard timeout** so the spinner can never hang permanently. Results are merged into the per-language cache, so each language is fetched at most once per assessment.

> **THE DESIGN DECISION THAT MAKES THIS TRUSTWORTHY**
>
> `classifiedPersonaId` is captured from the *first* successful classification and never updated afterwards. Switching language therefore re-generates the narrative, reasoning and recommendations in the new language but **cannot change which persona you are**. Without this, a user toggling EN → BM could be reclassified from Builder to Strategist, which would destroy credibility.

The rule-based fallback

If classification fails, `getFallbackPersona()` tallies the persona mapped to each MCQ answer and returns the highest count, defaulting to `explorer` on a tie or empty set. The Results screen then renders a yellow "rule-based" badge instead of the cyan "AI classified" badge, so the degradation is visible rather than silent.

6.2 Landing screen

A full-bleed hero image (`hero-v5.jpg`, referenced through `import.meta.env.BASE_URL` so it resolves correctly under the GitHub Pages sub-path), three feature cards (AI-Powered Classification, AI Personalised Learning Pathway, Workforce Analytics), four trust badges (AI-Driven, Employee-Centric, Secure & Trusted, Future-Ready), and three calls to action: start the assessment, view statistics, or open the contact form.

6.3 Question screen

Renders one MCQ at a time with a numeric progress indicator (`01` of `06`) and an animated progress bar. Selecting an option records the answer and waits **600 ms** before advancing, so the selection animation completes. After question five it advances to the open-question screen rather than the next question.

6.4 Open-question screen

Two-part form. **Part A** is the mandatory role field. **Part B** is the open-ended textarea with a **20-character minimum** enforced client-side; the submit button stays disabled until both a non-empty trimmed role and a sufficiently long trimmed response are present. On submit, the full answer array is assembled — five MCQs, then the role as `questionId 6`, then the open response — stored for later re-use, and posted.

6.5 AI thinking screen

A four-step animated sequence — **Observe** → **Reason** → **Decide** → **Produce** — each step displayed for **1,900 ms** with its own icon and colour, marking the previous step complete as it advances.

> **BE AWARE DURING THE DEMO**
>
> These steps are a **timed animation, not telemetry**. They are not tied to the actual LLM request, do not reflect real progress, and will complete after a fixed 7.6 seconds regardless of whether the model has responded. The screen transitions when the fetch resolves, not when the animation ends.

6.6 Results screen

Presents, in order: a classification badge showing either "AI classified · N% confidence" or the yellow rule-based fallback; a large persona hero with icon, name, tagline and either the AI narrative or the static localised description; an AI reasoning panel; the three personalised learning recommendations as numbered cards; a four-card reference grid of all personas with the user's own marked "YOU"; and three actions — retake, view full report, and open the HR dashboard.

6.7 Report screen

A single-page A4-shaped report card with a TM-branded header band, the persona badge and confidence, four dimension bars, the AI narrative and reasoning side by side, strengths and growth areas, the learning pathway, and a footer.

How the four dimension scores are produced

> **IN PLAIN WORDS — READ THIS BEFORE YOU PRESENT THE REPORT**
>
> The report shows four bars: AI Awareness, Practical Application, Strategic Thinking, Collaboration & Ethics. They look like four separate measurements. **They are not.** All four are calculated from the one confidence number, nudged up or down by a fixed amount depending on your persona.
>
> So if a judge or client asks "how did you score Strategic Thinking?", the honest answer today is "it's derived from the overall confidence score." Improvement #9 in Document 2 fixes this by asking the AI for four real scores instead.

The four dimensions — AI Awareness, Practical Application, Strategic Thinking, Collaboration & Ethics — are **derived arithmetically from the single confidence value**, not separately assessed. The formula is:

```
score = clamp(round(confidence × 100) + offset[persona][i], 52, 99)
```

with a fixed per-persona offset vector, for example `builder = [+2, +10, −4, +4]`, which tilts the profile toward Practical Application. The strengths and growth areas are likewise **static per-persona lists**, hand-translated into all three languages. They are not generated by the model.

PDF generation

Both download buttons call the same handler, which dynamically imports `html-to-image` and `jspdf` only on first use (keeping them out of the initial bundle), rasterises the report node at `pixelRatio: 2` on a `#0d1117` background, filters out any element using `backdrop-filter` because SVG foreign-object rendering cannot handle it, encodes to JPEG at quality 0.93, and stretches the image to exactly one A4 page to eliminate a phantom blank second page. The filename is derived from the user's role with all non-alphanumerics replaced by underscores.

The comment in the source explains why `html-to-image` was chosen over the also-installed `html2canvas`: it uses the browser's native SVG renderer rather than a custom CSS parser, so Tailwind v4's `oklch` colours, CSS variables and gradients survive.

6.8 HR Dashboard

> **IN PLAIN WORDS — THE MOST IMPORTANT THING TO UNDERSTAND HERE**
>
> The HR dashboard looks like a real team of thirteen people. **Twelve of them do not exist.** Their names, roles, grades, scores and trends were typed into the code. Only the thirteenth row — labelled "You (current)" — is a real assessment.
>
> Everything calculated *from* that team (completion rate, dominant persona, skills gaps, succession candidates, the 90-day plan) uses genuine formulas and a genuine AI call, but it is doing genuine maths on invented people. The logic is real; the population is not.

Four tabs under a persistent header. A standing "Human-in-the-Loop Governance" notice states that AI classifications require HR Manager review before being recorded in official systems.

Data source

The dashboard composes a team from a **hardcoded array of twelve demo employees** (ten complete, two pending, with TM-style grades F through I and roles such as Network Engineer, AI Governance and HR Learning & Development), to which the current real user is appended as a thirteenth member named `"You (current)"`, carrying their genuine persona and their confidence converted to a percentage score.

Tab 1 — Team Overview

- **Four KPI cards:** completion rate (`completed / total`), average score across completed assessments, the dominant persona with its count and share, and the pending count.
- **Persona distribution:** four clickable cards; clicking one filters the results table, clicking it again clears the filter. The highest-count persona receives a "Dominant" badge.
- **Individual results table:** Employee, Role, Grade, Persona, Score, Trend, Status. Scores are colour-banded at ≥85 green, ≥70 cyan, below 70 amber. The current user's row is highlighted and labelled.
- **Send Reminder (2):** flips to "✓ Reminder Sent" and disables. **No email, notification or API call occurs.**
- **AI Workforce Insight:** a localised paragraph selected by dominant persona, plus — when a real result exists — the user's actual AI recommendations shown as "pending your approval".
- **Approve & Route to Training Programme:** the governance action. It sets a single boolean in component state and changes the button to "Training Programme Activated — HR Approved". Nothing is persisted, nothing is routed, and a page refresh resets it.
- **Export CSV:** genuinely functional — builds a Blob, creates an object URL and triggers a download of `ainspire-team-report.csv` containing the demo rows plus the current user.

Tab 2 — Skills Gap Analysis

Four range sliders let HR set a target persona distribution, initialised to `{ explorer: 20, builder: 40, strategist: 25, visionary: 15 }`. A badge validates that the four values total exactly 100%; the sliders do not auto-rebalance.

The calculations are:

- **Overall readiness** = `(builder + strategist + visionary) / total × 100` — Explorers are deliberately excluded from the "AI-ready" definition.
- **Target readiness** = the sum of the three non-Explorer targets.
- **Uplift** = `max(0, targetReadiness − overallReadiness)`, also expressed as an estimated employee headcount.
- **Per-persona gap** = `target − current`, with headcount = `round(|gap| / 100 × total)`, displayed as a shortfall, an excess, or "✓ On target".

Each persona row carries a hardcoded recommended action, for example Builder → "LLM Engineering bootcamp, hands-on build sprints". Targets are never saved.

Tab 3 — Succession Planning

Candidates are selected automatically by rule: **score ≥ 80 and persona is Strategist or Visionary**. Each candidate is seeded into a pipeline stage (the first two into stage 2, the third into stage 3, the rest into stage 1) and is assigned a suggested AI leadership role — Visionaries become "AI Council Member / GCTO AI Champion", Strategists become "AI Programme Lead / AI Transformation Manager".

Three KPIs are shown: candidate count, the number at the advanced stages, and the average candidate score. Candidates advance via previous/next buttons and a dropdown in the profile table.

> **TWO THINGS TO KNOW BEFORE DEMONSTRATING THIS TAB**
>
> First, the heading says "drag status to advance candidates" but **there is no drag-and-drop implementation** — advancement is by button and select only. Second, all pipeline changes live in component state and are lost on navigation.

Tab 4 — Action Plan

The only tab that calls the API. It computes the live persona distribution from the team, derives a skills-gap array against the fixed target distribution, and posts to `/api/action-plan` with `divisionName` hardcoded to **"IT Strategy & Orchestration"**.

It renders a proper four-state machine — empty ("Ready to Generate"), loading (disabled button plus three animated skeleton cards), error ("Generation Failed" with the raw error string), and success. The success view renders the plan title, executive summary, three colour-coded phases (cyan, purple, amber) each with numbered actions showing owner, effort (green/amber/red for Low/Medium/High) and persona tags, a success metric per phase, the expected 90-day outcome, and key risks.

Print / Export Plan opens a new browser window, writes a complete standalone HTML document with its own stylesheet, and calls `window.print()` — the user can then choose "Save as PDF" in the print dialog. No PDF is produced directly and nothing is saved server-side.

6.9 Statistics screen

> **IN PLAIN WORDS — HANDLE THIS SCREEN WITH CARE**
>
> This is the most impressive-looking screen in the product and the least real. Every division, every chart, every course completion figure and every readiness score was typed into the code. **The screen makes zero calls to anything.**
>
> It is still worth showing — it demonstrates exactly what the product looks like at TM scale. Just describe it as an illustrative model, out loud, before anyone asks.

The largest screen in the application and the most visually impressive. It presents organisation-wide workforce intelligence in two sections across eighteen distinct views.

Data foundation

Ten Telekom Malaysia divisions are hardcoded with employee counts, completion counts, persona percentages that sum to 100, an AI readiness index and an average confidence: **GNT** (Group Network Technology, 2,100), **GCX** (Group Customer Experience, 1,850), **GITD** (Group IT & Digital, 1,200), **TM One** (890), **GF** (Group Finance, 650), **GHCM** (520), **TM Global** (420), **GIS** (Group Information Security, 380), **GP** (Group Procurement, 290) and **GS** (Group Strategy, 180). Totals are derived by reduction; the workforce target is a fixed `8,480` and organisation average confidence a fixed `74`.

Cross-filtering

A single `selectedDivision` state drives the whole screen. Clicking a bar in "Participation by Division" selects that division; clicking it again clears it. When selected, a filter banner appears, the hero counters switch to division figures, and the monthly trend is **scaled proportionally** by `division.completed / TOTAL_COMPLETED` — an estimate, not real division history. A custom `useCounter` hook animates headline numbers from zero over 1,400 ms in 16 ms ticks.

Section 1 — Workforce Analytics (views 1–11)

- **Hero** with animated total assessments against the 8,480 target, participation rate and this month's volume.
- **Four KPI cards:** assessments completed, AI-ready employees (`completed × (builder + strategist + visionary) / 100`), average confidence, and divisions assessed.
- **Monthly Assessment Trend** — Recharts area chart, cumulative plus monthly new.
- **Participation by Division** — the interactive bar chart that drives filtering.
- **AI Persona Distribution** — donut chart aggregating persona headcounts across all divisions.
- **AI Readiness Index** — ten divisions ranked, banded at ≥70 High, 60–69 Mid, below 60 Developing. Despite the tooltip describing a composite formula, these index values are **hardcoded constants, not calculated**.
- **Persona Mix by Division** — stacked percentage bar chart.
- **Cohort Benchmarking** — each division compared against a TM organisation average of `67` and a telco industry benchmark of `71`, both hardcoded.

Section 2 — Learning Pathway Analytics (views 12–18)

- **Learning Hours Logged** — per division, computed as completed assessments multiplied by a hardcoded per-division average-hours factor, against a target of 80% of headcount.
- **Certifications & Badges** — earned derived from a hardcoded per-division factor; target is 40% of completed; "on track" at 70% of target.
- **Monthly Completion Rate Trend** — line chart of actual rate against monthly target.
- **Course Performance** — six named courses (AI Fundamentals for Business, Prompt Engineering Essentials, Python for Data Science, AI Strategy & Leadership, Machine Learning Basics, Cybersecurity & AI) with enrolled, completed,

completion rate, drop-off rate and average days to finish.

- **Skill Coverage Heatmap** — a 10 × 7 matrix (divisions × skills: Prompting, Data Analysis, AI Strategy, ML Basics, Security & AI, Automation, AI Ethics) with levels 0–3 derived from persona percentages, for example `Prompting = min(3, round(builder / 25))`.
- **Persona-to-Course Alignment** — on-track versus off-track counts per persona against recommended courses.

> **DEMO INTEGRITY WARNING**
>
> This screen carries a badge reading **"✦ Live Workforce Intelligence · AiNspire"** while containing **no API calls whatsoever**. The only mitigation is a small footer line: "Data is simulated for demonstration purposes." If a judge or client reads the badge and not the footer, the claim is misleading. Say "illustrative" out loud when you present it.

6.10 Contact screen

A form capturing name, email, a purpose chosen from a custom dropdown, and a message. On submit it **constructs a `mailto:` link** to `ctsoo@tm.com.my` with a URL-encoded subject `[AiNspire] {purpose} — from {name}` and body, and opens it in a new window. There is no backend, no form storage and no delivery guarantee — if the user's machine has no mail client configured, nothing happens.

7 Cross-Cutting Mechanisms

Mechanism	How it works
Trilingual output	UI strings, questions and options come from a compiled 1,310-line dictionary. LLM output language is enforced by a conditional prompt directive demanding no English mixing.
Per-language result cache	Each language is classified at most once per assessment; results are keyed by language in a single state object.
Persona stability	The classified persona is frozen after the first success so language switching cannot change the outcome.
Graceful degradation	A plurality-vote fallback persona plus static localised copy keeps the app functional with the API entirely offline, with a visible yellow badge marking the degraded state.
Request de-duplication	A `useRef` tracks the in-flight language, and an `AbortController` with a 45-second timeout prevents a hung spinner on the language-switch path.
Client-side PDF	Dynamic imports of html-to-image and jsPDF, SVG-based rasterisation, single-page A4 stretch.
CSV export	Blob plus object URL plus synthetic anchor click — fully functional, no server involvement.
Print export	New window, injected standalone HTML document, `window.print()`.
Structured logging	Named events (`request_started` , `classify_started` , `classify_success` , `classify_failed` , `action_plan_*`) with duration tracking and header redaction.

8 Live vs Simulated — Reality Register

This table is the single most important page in this document. Keep it beside you when demonstrating to clients or judges.

HOW TO USE THIS PAGE

Read down the middle column. LIVE items you can claim freely and even invite someone to test on the spot. DERIVED / STATIC items you should describe accurately if asked, but they are not dishonest. SIMULATED / COSMETIC items must be introduced as illustrative **before** you show them — never after someone questions them.

Capability	Status	What is actually happening
Persona classification by LLM	LIVE	Real call to the TM `gpt-oss-20b` endpoint with the employee's genuine answers.
Confidence, reasoning, narrative	LIVE	Generated per submission by the model.
Three learning recommendations	LIVE	Generated per submission; free-text titles, not from a TM course catalogue.

Capability	Status	What is actually happening
Trilingual AI output	LIVE	Re-generated per language by the model on demand.
90-day action plan	LIVE	Real LLM call — but computed from the demo team, not real employees.
Report PDF download	LIVE	Genuinely produced in the browser and saved to disk.
CSV export	LIVE	Genuine file download of the demo team plus the current user.
Rule-based fallback	LIVE	Real plurality-vote logic with an honest visual indicator.
Four dimension scores on the report	DERIVED	Arithmetic transform of the single confidence value, not four measurements.
Strengths & growth areas	STATIC	Fixed per-persona lists, hand-translated into three languages.
"AI is observing / reasoning / deciding"	ANIMATION	A fixed 4 × 1.9-second timer with no link to the real request.
HR team of 12 employees	SIMULATED	Hardcoded array. Only the thirteenth member — "You (current)" — is real.
Scores, trends, grades in the HR table	SIMULATED	Literal values such as `score: 84, trend: '+12'`.
Send Reminder	COSMETIC	Sets a boolean. No email or notification is sent.
Approve & Route to Training	COSMETIC	Sets a boolean. Nothing is persisted or routed; resets on refresh.
Succession pipeline	VOLATILE	Real selection rule over simulated people; stage changes are lost on navigation.
Skills-gap targets	VOLATILE	Real arithmetic over simulated headcount; targets are never saved.
All ten divisions and their figures	SIMULATED	Hardcoded constants; the "Live Workforce Intelligence" badge is not accurate.
Learning hours, certifications, courses	SIMULATED	Derived from hardcoded factors; no LMS or TM Academy integration exists.
AI Readiness Index	SIMULATED	Hardcoded per division despite a tooltip describing a formula.
Contact form	PARTIAL	Opens the user's mail client via `mailto:`; no backend, no delivery guarantee.
Any data persistence at all	NONE	The Drizzle schema is empty. Nothing — not one assessment — is stored anywhere.
Authentication / SSO	NONE	No login. Anyone with the URL reaches the HR dashboard.

9 Defect & Gap Register

Confirmed by direct code inspection, ordered by severity.

THE THREE THAT ACTUALLY MATTER

#1 — a live TM password for the AI service is sitting in public code. Fix today, regardless of the hackathon.
#2 — there is no login, so anyone with the link can open the HR dashboard.
#3 — nothing is ever saved, which is why the HR dashboard has to use invented people.

Fix those three and most of the others become much smaller problems. Document 2 turns this list into an assigned plan.

#	Severity	Finding	Detail and impact
1	CRITICAL	TM LLM API key committed to a public repository	A live key is the hardcoded default in `lib/llm.ts`, present on `main` and throughout git history. Must be rotated, removed from code, and purged from history. This is a disclosable security incident, not a code-quality issue.
2	CRITICAL	No authentication or authorisation	The HR dashboard — including individual employee results and succession candidates — is reachable by anyone with the public URL. For a real TM deployment this is a blocking compliance failure.
3	CRITICAL	Zero persistence	`lib/db/src/schema/index.ts` defines no tables. Every assessment is lost on refresh. The entire HR analytics proposition is therefore unbacked.
4	HIGH	Open CORS policy	`cors()` with defaults allows every origin to call the classification and action-plan endpoints, which consume TM LLM quota.
5	HIGH	No rate limiting or abuse protection	Both AI endpoints are unauthenticated and unthrottled. A trivial loop can exhaust TM GPU capacity and budget.
6	HIGH	No timeout or retry on the LLM call	`chatComplete` has no `AbortSignal`. A stalled proxy holds the Express request open indefinitely; the initial classification has no client-side timeout either.
7	HIGH	`/action-plan` has no input validation	Calls `skillsGap.map` before any guard, throwing a `TypeError` on a malformed body and returning an opaque 500.
8	HIGH	Error details leaked to the client	Both routes return `details: String(err)`, which on an upstream failure includes the raw LLM response body and may expose internal endpoint information.
9	HIGH	Misleading "Live Workforce Intelligence" badge	The statistics screen makes a liveness claim it cannot support. Material reputational risk in a client or judging setting.
10	MEDIUM	LLM response is parsed but never schema-validated	Three ad-hoc corrections are applied, but a structurally wrong object — missing `narrative`, or recommendations lacking `title` — passes straight into the renderer. The generated Zod schemas that would prevent this already exist and are unused.

#	Severity	Finding	Detail and impact
11	MEDIUM	Division-by-zero exposure in skills-gap maths	Readiness calculations divide by the completed-member count without a guard; an empty team yields `NaN` in the UI.
12	MEDIUM	OpenAPI specification is stale and wrong	Omits `/action-plan` entirely, omits the `lang` request field, and still states the classifier is "Claude" when it is `gpt-oss-20b`.
13	MEDIUM	UI promises behaviour that does not exist	"drag status to advance candidates" has no drag implementation; "Send Reminder" sends nothing; "Approve & Route to Training Programme" routes nothing.
14	MEDIUM	Division name hardcoded in the action-plan request	Always submits "IT Strategy & Orchestration" regardless of the team actually being viewed.
15	LOW	Dead and duplicated dependencies	`api-client-react` is entirely unused; both `html2canvas` and `html-to-image` are installed though only the latter is used; `wouter` is installed but no routing exists.
16	LOW	Replit build artefacts still in the tree	Duplicate `"… 2.mjs"` files in `dist`, a `pnpm-lock 2.yaml`, `.replit` config, a `gitsafe-backup` remote, and an unpushed `replit-agent` branch.
17	LOW	Contact form has no reliable delivery	Depends entirely on a configured local mail client; silently does nothing otherwise.

10 Prototype Backlog — `mockup-sandbox`

A separate Vite application auto-discovers component files under `src/components/mockups/` and renders them in a preview shell. It holds six design prototypes that were never integrated into the product. They are, in effect, an already-made product roadmap — and they map almost exactly onto the gaps listed in Section 9.

Prototype	Addresses gap	What it designs
`sso-login`	#2 No auth	A TM single-sign-on entry screen — the missing identity layer.
`persistent-db`	#3 No persistence	A records view keyed by TM employee ID (`TM-048391`), with division, grade, persona, score, assessment date, quarterly cycle and status. Effectively the missing database schema, already designed.
`hr-portal`	HR UX	A refined HR team view with named employees, grades, personas and score trends.
`division-analytics`	Analytics	Eight TM divisions with completion, average score, top persona and a readiness figure, plus a monthly trend.
`pdf-report`	Report quality	A light-themed print-optimised report including a **named learning catalogue with providers** — LinkedIn Learning, **TM Academy**, Coursera — with duration, level and badges such as "TM Exclusive". This is the design for turning free-text AI recommendations into real, bookable courses.
`landing-redesign`	Marketing	An alternative landing layout.

> **STRATEGIC OBSERVATION**
>
> Every major gap in the live product already has a corresponding design in the sandbox. The team is not missing a vision — it is missing the integration work. Document 2 converts this observation into a prioritised plan with named owners.